Bangladesh University of Engineering and Technology (BUET)

Department of Computer Science and Engineering

Generating All Triangulations of Biconnected Plane Graphs in Amortized Constant Time per Triangulation

Undergraduate Thesis

Tawkir Aziz Rahman

Dhaka-1000, Bangladesh

June 3, 2026

Abstract

We present an algorithm for generating all triangulations of a biconnected plane graph G with n vertices in $O(1)$ amortized time per triangulation and $O(n)$ space. The method builds a two-layer structure: a local genealogical tree of triangulations for each face and a global depth-first search (DFS) that combines those face triangulations without creating multi-edges. The key idea is a safe root vertex selection that prevents conflicts across faces, together with a pruning strategy and a valid generating set that keep each flip constant-time. The thesis revisits the cycle triangulation algorithm of Parvez, Rahman, and Nakano [**?**], identifies why it fails on biconnected graphs, and then shows how to extend the approach while preserving the amortized $O(1)$ bound. We also provide empirical validation using the measurements recorded in `results.csv`.

Contents

List of Figures

List of Algorithms

Chapter 1

Introduction

To achieve our objective of generating all triangulations for biconnected plane graphs, we first study the closest previous approaches. The most relevant work is the 2011 JGAA paper by Parvez, Rahman, and Nakano [**?**]. In that work, the authors present an algorithm that generates all triangulations of a cycle in amortized constant time per triangulation using $O(n)$ space, where n is the number of vertices in the cycle. Their main idea is to organize triangulations into a genealogical tree whose nodes are canonical triangulations, and to define a unique parent-child relationship by a local flip rule.

That cycle algorithm is one of the main parts of our method. However, our target is broader: we want to generate all triangulations of biconnected plane graphs. The triconnected case in [**?**] can treat faces independently because multi-edges cannot occur. In biconnected graphs, separation pairs appear and allow the same chord to arise in two different faces, creating the multi-edge problem. Therefore, the main technical challenge is to preserve the constant-time flip structure while preventing multi-edges across faces.

In this thesis, we keep the author's algorithmic intuition and expand it into a full thesis structure with formal definitions, proofs, and experiments. The flow is as follows. Chapter 2 provides preliminaries. Chapter 3 revisits the cycle triangulation algorithm. Chapter 4 explains the distinction between the original generating chord notion and our generating set. Chapters 5 and 6 describe why the triconnected algorithm fails for biconnected graphs and introduce the face-by-face DFS approach. Chapters 7 and 8 develop the pruning strategy and safe root theory. Chapter 9 introduces the valid generating set and constant-time updates. Chapter 10 presents the full algorithm. Chapter 11 analyzes time and space complexity, Chapter 12 reports experiments from `results.csv`, Chapter 13 discusses one-connected graphs, and Chapter 16 concludes.

Chapter 2

Preliminaries

We define the terms used throughout the thesis. Let $G = (V, E)$ be a connected simple graph with vertex set V and edge set E. An edge connecting vertices v_i and v_j is denoted by (v_i, v_j) where $v_i \neq v_j$. The connectivity of a graph is the minimum number of vertices whose removal disconnects the graph. A graph is *biconnected* if it remains connected after removing any single vertex, and *triconnected* if it remains connected after removing any two vertices [**?**, **?**].

A graph is *planar* if it can be embedded in the plane so that no edges cross except at their endpoints. A *plane graph* is a planar graph with a fixed embedding. This embedding divides the plane into regions called faces. The unbounded region is the outer face; all other regions are inner faces. A plane graph is a *plane triangulation* if each face is a triangle. Note that edges are not required to be straight line segments.

A *cycle* C in a plane graph is a sequence of distinct vertices $v_1, v_2, \ldots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < k$ and $(v_k, v_1) \in E$. A face boundary is such a cycle. A *chord* of a cycle is an edge between two non-consecutive vertices that lies inside the face. A set of non-intersecting chords that decomposes the cycle into triangles is a *triangulation* of the cycle.

(a) Triangulation A (b) Triangulation B

Figure 2.1: Two distinct triangulations of a cycle

In a triangulation T of a cycle, the set of chords is maximal: any chord not in T would intersect a chord in T. We say a vertex v_j is *visible* from v_i if the chord (v_i, v_j) is present in the triangulation.

Throughout this thesis, we work with labeled graphs, so each vertex has a unique

label. This assumption matches the algorithmic setting of [**?**] and the experimental data in `results.csv`.

Chapter 3

Triangulating a Cycle in Amortized Constant Time

We first revisit the cycle triangulation algorithm from [**?**] because it is the backbone of our approach. Let C be a cycle with vertices $v_1, v_2, \ldots, v_n$ in counterclockwise order. We build a genealogical tree of triangulations in which each node is a triangulation and each edge corresponds to a single chord flip. The parent-child relationship is defined using blocking chords and a leftmost rule, which yields a unique path from the root triangulation to every other triangulation.

3.1 Root vertex and visibility

We pick a root vertex, say v_1, and classify chords according to visibility from v_1. If every non-neighboring vertex is connected to v_1, then all vertices are visible and the triangulation is a root triangulation.

Definition 3.1 (Blocking chord)**.** A chord (v_i, v_j) is a *blocking chord* with respect to root v_1 if it does not have v_1 as an endpoint and is visible from v_1, meaning it forms a triangle with v_1 and blocks visibility to some vertices behind it.

Definition 3.2 (Generating chord)**.** A chord incident to the root v_1 is called a *generating chord*. In the cycle algorithm of [**?**], a generating chord is flippable if its flip becomes the leftmost blocking chord of the child triangulation.

3.2 Chord flipping

A flip is defined on a quadrilateral. Suppose a triangulation contains a chord (b, d) inside a quadrilateral a, b, c, d in order around the boundary. The flip removes (b, d) and adds the other diagonal (a, c). This is a constant-time operation and changes only one chord.

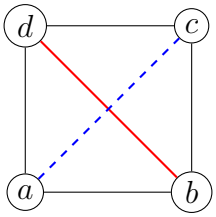

Figure 3.1: Flipping chord (b, d) to (a, c) in a quadrilateral

3.3 Leftmost blocking chord

To ensure a unique parent, [**?**] defines the *leftmost blocking chord*. Intuitively, among all blocking chords, the one that blocks the leftmost blocked vertex is selected. The parent of a triangulation is obtained by flipping this leftmost blocking chord.

Lemma 3.3 (Uniqueness of leftmost blocking chord)**.** *If a triangulation has at least one blocking chord, then it has a unique leftmost blocking chord.*

Proof. Suppose there are two distinct blocking chords that both block the leftmost blocked vertex. The root vertex must see each blocking chord to classify it as blocking, so each blocking chord forms a triangle with the root. If two blocking chords both block the same leftmost vertex, then one of them must lie closer to the root. That chord blocks visibility to the other, contradicting that both are visible from the root. Therefore, the leftmost blocking chord is unique. □

3.4 Parent-child relationship

A triangulation is the parent of another if flipping the child's leftmost blocking chord produces the parent. By the uniqueness above, each triangulation has exactly one parent (except the root). Thus the genealogical tree has a unique path from the root to any node.

When we keep flipping leftmost blocking chords, we eventually reach a triangulation with no blocking chord, meaning all chords are incident to the root. This is the root triangulation.

3.5 Generating children

To generate children of a triangulation, we flip generating chords. The key observation from [**?**] is that only certain generating chords are eligible: if $(v_r, v_{r'})$ is the leftmost blocking chord of a triangulation, then flipping a generating chord (v_1, v_j) can create a child whose leftmost blocking chord is the new chord only when $j \geq r$.

Lemma 3.4 (Flippable generating chords)**.** *Let $(v_r, v_{r'})$ with $r < r'$ be the leftmost blocking chord of a triangulation T. Then flipping (v_1, v_j) results in a child whose new chord is the leftmost blocking chord if and only if $j \geq r$.*

Proof. If $j < r$, then the flip stays inside the region bounded by $(v_1, v_{r'})$, so the leftmost blocking chord $(v_r, v_{r'})$ remains visible and leftmost, and the new chord cannot become the leftmost blocking chord. If $j \geq r$, the flipped chord lies to the left of $(v_r, v_{r'})$ in the visibility order, so the new chord is visible from the root and blocks a vertex no farther right than $v_{r'}$. Therefore, it becomes the leftmost blocking chord of the child. □

3.6 Data structures and complexity

The cycle algorithm stores the triangulation using three sets:

- GS: generating chords incident to the root.
- O: opposite pairs used to perform constant-time flips.
- L: all remaining chords of the triangulation.

Since the cycle has n vertices, each set is $O(n)$ in size. Each flip changes only a constant number of chords, so moving to a child triangulation takes $O(1)$ time. The depth of any branch is at most $n - 2$, so recursion uses $O(n)$ stack space. Therefore, the algorithm runs in amortized $O(1)$ time per triangulation with $O(n)$ space.

3.7 Cycle triangulation algorithm

Algorithm 1 summarizes the cycle enumeration procedure used in the triconnected case and adapted later in this thesis.

Algorithm 1 EnumerateCycleTriangulations

procedure EnumerateCycleTriangulations(C)
    Choose root vertex v_1
    Build root triangulation by adding all chords (v_1, v_j) for non-neighboring v_j
    Initialize GS, O, and L
    DFSLocal(T_{root})
end procedure

procedure DFSLocal(T)
    Output T
    if T has a leftmost blocking chord $(v_r, v_{r'})$ **then**
        for each generating chord (v_1, v_j) with $j \geq r$ **do**
            Flip (v_1, v_j) to obtain T'
            DFSLocal(T')
        end for
    end if
end procedure

Chapter 4

Distinction in Generating Set Definition

The cycle algorithm in [**?**] calls only those chords flippable if their flip becomes the leftmost blocking chord. In our thesis, we distinguish two sets:

- GS: all chords incident to the root (every chord with one endpoint at the root).
- VGS: the valid subset of GS whose flips do not create multi-edges with previously processed faces.

This distinction is necessary because in biconnected graphs a generating chord may lead to a multi-edge in another face. We therefore maintain a separate valid generating set that is updated after each flip.

Chapter 5

Why the Triconnected Algorithm Fails for Biconnected Graphs

Parvez, Rahman, and Nakano [**?**] extend the cycle algorithm to triconnected plane graphs by triangulating all faces independently. The key reason this works is that triconnected graphs have no separation pairs, so a chord added in one face cannot also appear in another face. Consequently, no multi-edge can occur.

In biconnected graphs, separation pairs exist. A separation pair (a, b) splits the graph into two components if removed. The two components can each place a chord between a and b on different faces. If we triangulate those faces independently, the chord (a, b) may appear twice, creating a multi-edge. This invalidates the assumption of independent face triangulation.

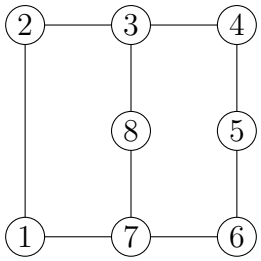

Figure 5.1: A biconnected graph where $(3, 7)$ is a separation pair. The chord $(3, 7)$ can appear in two faces.

This multi-edge issue is the core reason we cannot directly apply the triconnected algorithm to biconnected graphs. We must process faces in an order that respects chords already chosen, and we must prune invalid branches early.

Chapter 6

Processing Faces One by One

To avoid multi-edges, we process faces one by one using a DFS across faces. For each face F_i, we generate its local genealogical tree and traverse it. For every triangulation of F_i, we recursively process the next face, carrying a global chord set S that tracks all chords chosen so far.

Consider faces $F_1, F_2, \ldots, F_k$. For each triangulation $T_{1,j}$ of F_1, we enumerate all triangulations $T_{2,*}$ of F_2 that do not conflict with $T_{1,j}$. For each compatible pair, we move to F_3, and so on, until we triangulate F_k. Each root-to-leaf path corresponds to a complete triangulation of G.

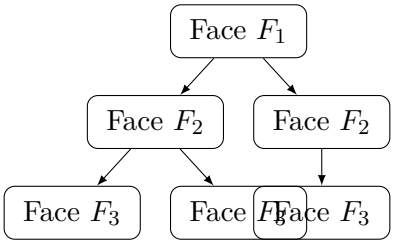

Figure 6.1: DFS traversal across faces. Each branch corresponds to one compatible choice per face.

This layered DFS preserves the author's intended flow: we keep the local genealogical trees from the cycle algorithm but combine them in a global search that respects already chosen chords.

Chapter 7

Pruning Strategy and Conflict Tracking

A naive approach would generate all local triangulations and then filter invalid ones. This is too expensive because invalid triangulations could dominate the search and destroy the amortized constant-time bound. Instead, we prune branches as soon as a conflicting chord appears.

7.1 Persistence of non-root chords

In the cycle algorithm, we only flip generating chords (those incident to the root). Therefore, once a chord is introduced that is not incident to the root, it is never flipped again in the descendant triangulations.

Lemma 7.1 (Chord persistence)**.** *Let T be a triangulation of a face with root vertex v_1. If a chord (a, b) does not have v_1 as an endpoint, then (a, b) appears in every descendant triangulation of T in the local genealogical tree.*

Proof. Descendants are obtained only by flipping generating chords incident to v_1. A chord (a, b) with neither endpoint v_1 is never selected for flipping, and thus cannot be removed. Therefore it persists in all descendants. □

This property justifies pruning: if a newly flipped chord creates a multi-edge with any chord already in S, then every descendant triangulation will also contain this conflicting chord and therefore will be invalid. We can safely prune that branch without losing any valid triangulation.

7.2 Global chord set

We maintain a global chord set S (implemented as a hash map) that stores all chords from previously processed faces and the permanent edges of the graph. Each time we move to a new triangulation of face F_i, we insert its chords into S. When we backtrack, we remove them. This supports constant-time conflict checks for each flip.

Chapter 8

Safe Root Vertices

The pruning strategy assumes that the root triangulation of each face is valid. Therefore, we must guarantee that every face has a *safe root* vertex such that the fan triangulation from that root does not conflict with S.

8.1 Existence of safe roots

Theorem 8.1 (Existence of safe roots)**.** *For any face F_i with $n \geq 3$ vertices in a biconnected plane graph, and any global chord set S from previously processed faces, there exist at least two safe root vertices in F_i.*

Proof. If no chord in S has both endpoints on the boundary of F_i, then every vertex is safe. Otherwise, let (v_i, v_j) be a chord in S with endpoints on the boundary. By planarity, this chord separates the boundary into two disjoint subpaths. Any external chord must have both endpoints inside the same subpath; chords cannot cross (v_i, v_j). Therefore, the boundary is partitioned into two structures, each of which is a path with an external chord between its endpoints. We call each such subpath a T_n structure.

For T_1, the middle vertex has no non-neighboring vertex in the structure, so it is safe. For T_2, if one interior vertex has an external chord, the other interior vertex becomes a T_1 case and is safe. By induction on n, each T_n structure has at least one safe vertex. Since there are at least two such structures, there are at least two safe roots in F_i. □

8.2 Minimum number of valid triangulations

The safe root theorem implies at least two distinct fan triangulations for each face. Those two root triangulations differ in at least $n - 4$ chords, so the genealogical tree contains at least $n - 3$ distinct triangulations reachable by flips. Therefore, each face has at least $\Omega(n)$ valid triangulations, which is enough to amortize the $O(n)$ time for building the root triangulation and valid generating set.

8.3 Finding a safe root in $O(n)$ time

We can find a safe root using a two-pointer scan along the boundary. Let the vertices be $v_1, \ldots, v_n$ in order. We initialize $i = 1$ and $j = n - 1$ (neighbors are excluded). If $(v_i, v_j) \in S$, then v_i cannot be a safe root and we advance i. Otherwise, we decrease j because v_i still may be safe and we need to test shorter chords. This scan ends when $i + 1 = j$, at which point v_i is safe.

Algorithm 2 FindSafeRoot

```
1: procedure FindSafeRoot(F_i, S)
2:     Let v_1, ..., v_n be the boundary of F_i
3:     i ← 1, j ← n − 1
4:     while j > i + 1 do
5:         if (v_i, v_j) ∈ S then
6:             i ← i + 1
7:         else
8:             j ← j − 1
9:         end if
10:    end while
11:    return v_i
12: end procedure
```

The loop advances at least one pointer per step, so the time is $O(n)$. This matches the $O(n)$ cost of building the root fan triangulation and preserves the amortized constant-time bound.

Chapter 9

Valid Generating Set and Constant-Time Updates

We now introduce the valid generating set (VGS), which contains exactly those generating chords whose flips do not create multi-edges with S. During the initial root construction we compute VGS by checking each generating chord's opposite pair against S.

After each flip, only the opposite pairs of the four boundary edges of the quadrilateral change. Therefore, we only need to update the validity of those four chords. There are four cases:

1. The chord was valid and becomes invalid after the flip (remove from VGS).
2. The chord was invalid and becomes valid after the flip (insert into VGS).
3. The chord remains valid.
4. The chord remains invalid.

If VGS is stored as a linked list (or a hash set with pointers), each insert or delete is $O(1)$. Therefore, each flip still runs in $O(1)$ time while maintaining a correct set of valid generating chords.

Chapter 10

Full Algorithm

We now present the full algorithm in pseudocode. The main procedure processes faces in order, using the safe root algorithm for each face and enumerating valid triangulations with the local genealogical tree and VGS updates.

Algorithm 3 FindAllTriangulations

```
procedure FindAllTriangulations(G)
    Label faces F_1, F_2, ..., F_k
    Initialize global chord set S with all permanent edges
    Initialize empty partial triangulation T
    ProcessFace(1, T, S)
end procedure
```

Algorithm 4 ProcessFace

```
1: procedure ProcessFace(i, T, S)
2:     if i > k then
3:         Output T
4:         return
5:     end if
6:     r ← FindSafeRoot(F_i, S)
7:     Build root triangulation T_i of F_i using r
8:     Initialize GS, O, L, and VGS for T_i
9:     EnumerateFace(F_i, T_i, T, S)
10: end procedure

11: procedure EnumerateFace(F_i, T_i, T, S)
12:     Add chords of T_i to S
13:     ProcessFace(i + 1, T ∪ T_i, S)
14:     Remove chords of T_i from S
15:     for all chords e ∈ VGS do
16:         Flip e to obtain T_i'
17:         Update opposite pairs and VGS for the four boundary edges
18:         if new chord not in S then
19:             EnumerateFace(F_i, T_i', T, S)
20:         end if
21:     end for
22: end procedure
```

This pseudocode follows the author's flow: each face has its own genealogical tree, the DFS across faces is the outer layer, and pruning is enforced by the global chord set.

Chapter 11

Complexity Analysis

11.1 Time complexity

For each face, building the root triangulation and the initial valid generating set costs $O(n)$. By Theorem 8.1 and the argument in Chapter 8, each face has at least $\Omega(n)$ valid triangulations, so this $O(n)$ cost is amortized to $O(1)$ per triangulation within the face.

Each flip updates only a constant number of chords and opposite pairs, so the transition from one triangulation to the next takes $O(1)$ time. The outer DFS across faces does not introduce additional asymptotic overhead because each complete triangulation corresponds to exactly one leaf in the recursion tree and is generated by a constant amount of work per flip. Therefore, the overall algorithm runs in $O(1)$ amortized time per triangulation.

11.2 Space complexity

We store the current face's GS, VGS, O, and L sets, each of which is $O(n)$ for a face with n vertices. The global chord set S stores all permanent edges and all chords along the current recursion path, which is $O(n)$ because the number of edges in a plane graph is linear in n. The recursion depth across faces is also $O(n)$ since the number of faces is linear. Therefore, the total space complexity is $O(n)$.

Chapter 12

Experimental Validation

This section reports empirical validation using the latest measurements in `../time_complexity/results.csv`. Each row records: the instance name, number of vertices, number of triangulations, mean time, median time, minimum and maximum time, standard deviation, per-triangulation time in nanoseconds, peak memory, and memory per vertex. We include a sample table in the main text and provide the full dataset in Appendix A.

12.1 Sample summary table

12.2 Per-triangulation time plot

The measurements confirm that per-triangulation time remains small across a wide range of instances. The full dataset is included in the appendix for reproducibility.

Chapter 13

Why the Algorithm Does Not Extend to 1-Connected Graphs

In a 1-connected plane graph, a separation vertex can appear multiple times on the boundary of a single face. This can create faces in which the same vertex repeats in the boundary cycle. In such a case, no safe root may exist. For example, in the path graph 1-2-3-4, the unique face boundary is 1-2-3-4-3-2-1. Any root choice creates a repeated chord or a self-loop, so no safe root triangulation exists. Hence the algorithm cannot be directly extended to all 1-connected graphs.

However, some 1-connected graphs do admit safe roots. For example, in the face 1-2-3-4-5-2-1, vertices 1, 3, and 5 can be safe roots, and the algorithm still works. Therefore, the limitation is not absolute; it depends on the existence of safe roots in all faces.

Chapter 14

Limitations

The current algorithm is limited to biconnected plane graphs because the existence of safe roots is not guaranteed in general 1-connected graphs with repeated vertices on face boundaries. The pruning strategy also relies on the persistence of non-root chords, which fails if a face boundary is not a simple cycle.

Chapter 15

Future Work

Future work includes characterizing which 1-connected graphs admit safe roots in all faces and extending the pruning strategy to tolerate a controlled number of invalid generating chords. Another direction is to explore 1-planar graphs, where at most one chord can cross another. Extending the genealogical tree approach to that setting could lead to new enumeration algorithms.

Chapter 16

Conclusion

We presented an algorithm that enumerates all triangulations of a biconnected plane graph in $O(1)$ amortized time per triangulation and $O(n)$ space. The algorithm preserves the canonical genealogical structure of the Parvez-Rahman-Nakano cycle algorithm while resolving the multi-edge issue introduced by separation pairs. The key ideas are safe root selection, a valid generating set, and a pruning strategy based on the persistence of non-root chords. Empirical results from the latest dataset confirm the practical efficiency of the approach.

Appendix A

Full Experimental Results